// record push in set

st.insert (make_pair(dist [neighbour.first], neighbour.first));

}

}

}

return dist;

}

S.C = O(V+E)

T.C = O(E log V)

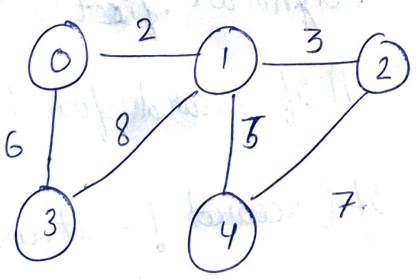
---

lec96 **Minimum Spanning Tree**

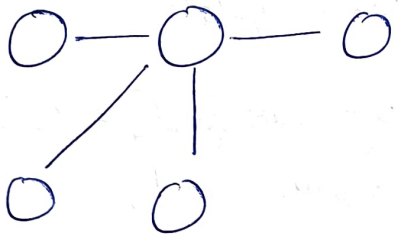
– **Prims Algorithm**

MST → when you convert a graph to a tree of 'n' nodes & 'n-1' edges.

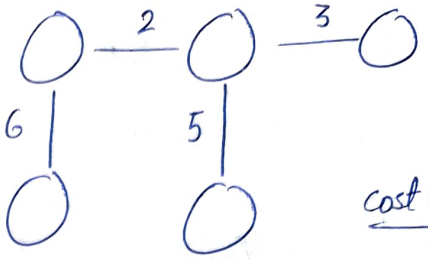
e.g. G =



spanning Tree 1 :

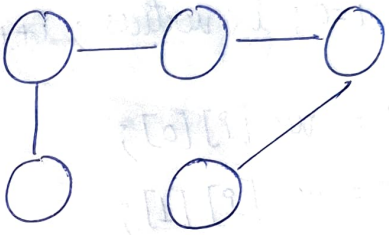


Spanning Tree 2: (MST) min weight.

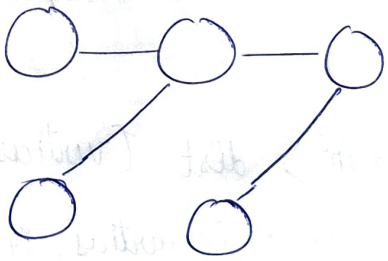


cost = 16

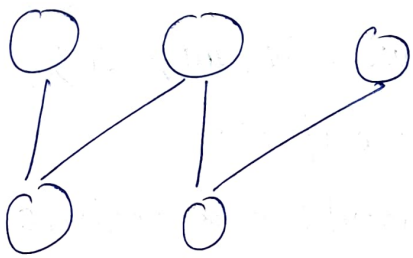
spanning Tree 3



spanning Tree 4



spanning Tree 5



.....

Prims Algorithm

Data structures : 3 arrays of size 'n'

0	∞	∞	∞	∞
0	1	2	3	4

Key

F	F	F	F	F
0	1	2	3	4

mst

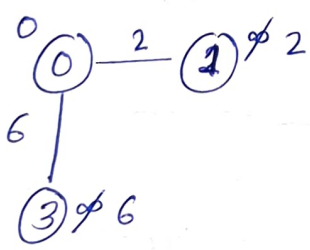
-1	-1	-1	-1	-1
0	1	2	3	4

parent.

step 1: Finding min in Key array (i.e '0') u → 0 (with mst = F)

step 2: mark true in mst (mst[u] = T).

step 3: find all adj nodes of u.



Key

0	2	∞	6	∞
0	1	2	3	4

update key array & Parent array

Parent

-1	0	-1	0	-1
0	1	2	3	4

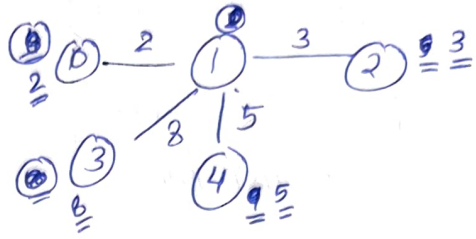
mst.

T	F	F	F	F
0	1	2	3	4

2nd pass

u → 0 (but mst[v] != F)

u → 1 ✓



Update all

Key

~~0~~	~~2~~	~~3~~	~~6~~	~~5~~
0	1	2	3	4

mst

T	T	F	F	F
0	1	2	3	4

Parent

-1	0	1	0	1
0	1	2	3	4

Key

0	2	3	6	5

3rd pass ..... till all mst is T.

code

```
vector<pair<int, pair<int,int>>>
Calculate PrimsMST (int n, int m,
vector<pair<pair<int,int>, int>> &g) {

    // create adj list
unordered_map<int, list<pair<int,
                    int>>> adj;
for (int i=0; i<g.size(); i++) {
    int u = g[i].first.first;
    int v = g[i].first.second;
    int w = g[i].second;
    adj[u].push_back(make_pair(v,w));
    adj[v].push_back(make_pair(u,w));
}
```

```
vector<int> key(n+1);
vector<bool> mst(n+1);
vector<int> parent(n+1);

for (int i=0; i<=n; i++) {
    key[i] = INT_MAX;
    parent[i] = -1;
    mST[i] = false;
}

// algo start
key[1] = 0;
parent[1] = -1;
for (int i=1; i<n; i++) {
    int mini = INT_MAX
    int u;
// find the min wali node
    for (int v=1; v<=n; v++) {
        if (mst[v] == false &&
            key[v] < mini) {
            u = v;
            mini = key[v];
        }
    }
// mark min node as true
mst[u] = true;
// check its adj nodes.
for (auto it : adj[u]) {
    int v = it.first;
    int w = it.second;
    if (mst[v] == false && w <
        key[v]) {
        parent[v] = u;
        key[v] = w;
    }
}
}

vector<pair<pair<int,int>, int>> result;
for (int i=2; i<=n; i++){
    result.push_back({{parent[i], i}, key[i]});
}
return result;
}
```

T.C = $O(n^2)$

Lec 97 **Disjoint Set**

2 operations (imp)

1) findParent() / findSet()
2) Union() or UnionSet().

Union by Rank & Path Compression.

→ connects disconnected components of graphs.

rank

x	0	0	0	0	0	0	
0	1	2	3	4	6	7	...

(1) (2) (3) ... (7)

Parent(2) = 2
(3) = 3
(1) = 1
:
(7) = 7

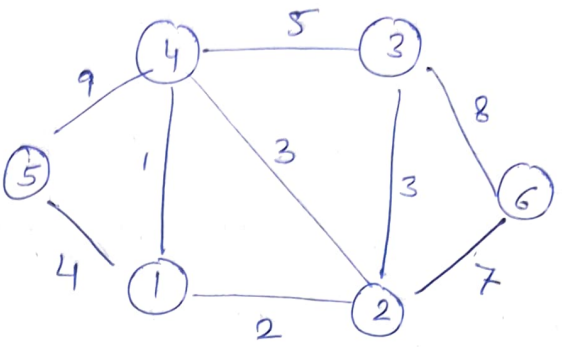
Union(i, j)

Same rank i, j

1) parent[i] = j (make any)
   rank[j]++;

if different i > j (say)

 # Kruskal Algorithm



same component? → check their parents

*) linear Data structure including all edges & sorted according to wt.

proceed ↓

wts	u	v
1	1	4
2	1	2
3	2	3
3	2	4
4	1	5
5	3	4
7	2	6
8	3	6
9	4	5

check if same parent

YES → ignore.

NO → merge them.

NOTE : It proceeds with edges increasing order

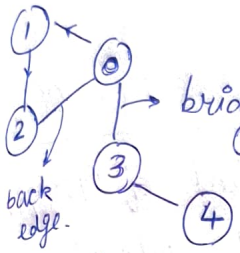
code ——— Disjoint Set code ———

```
void makeSet(vector<int> &parent,
vector<int> &rank, int n) {
    for(int i=0; i<n; i++) {
        parent[i] = i;
        rank[i] = 0;
    }
}

int findParent(vector<int> &
parent, int node) {
    if(parent[node] == node) {
        return node;
    }
    return parent[node] = findParent
        (parent, parent[node]);

void unionSet(int u, int v,
vector<int> &parent) &
vector<int> &rank) {
    u = findParent(parent, u);
    v = findParent(parent, v);
    if(rank[u] < rank[v]) {
        parent[u] = v;
    else if(rank[u] > rank[v]) {
        parent[v] = u;
    else {
        parent[v] = u;
        rank[u]++;
    }
}
```

```
int minimumSpanTree(vector<
vector<int>> &edges, int n) {
    sort(edges.begin(), edges.end())

    vector<int> parent(n);
    vector<int> rank(n);
    makeSet(parent, rank, n);
    int minWeight=0;
    for(int i=0; i<edges.size(); i++) {
        int u = findParent(parent,
                edges[i][0]);
        int v = findParent(parent,
                edges[i][1]);
        if(u!=v) {
            minWeight += wt;
            unionSet(u,v,parent,rank);
        }
    }
    return minWeight;
}
```

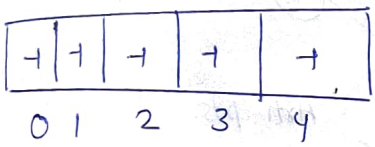
Bridge in GRAPH

- if removing an edge increases no. of components of graph then that edge is called bridge.

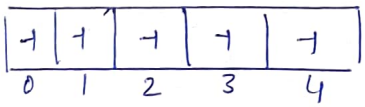


approach

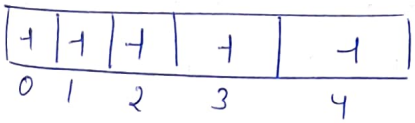
discovery



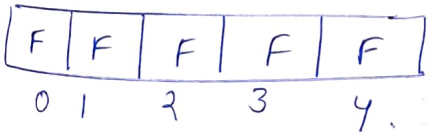
low (lowest possible time to discover)



parent



visited



back edge

↳ earliest possible time to reach a node can be updated.

low [node] = min (low [node], disc [neighbour])

To check bridge condtⁿ.

low [neighbour] > disc [node]

↳ if satisfies then bridge edge.

code

```
vector < vector <int> > findBridges
(vector < vector < int >> & edges,
 int v, int e) {
    // creati adj list.
    :
    :
    .
    -

 int timer = 0;
 vector <int> disc (v);
 vector <int> low (v);
 int parent = -1;
 unordered_map <int, bool> vis;
 for (int = 0; i<v; i++) {
        disc [i] = -1;
        low [i] = -1;
 }

 vector < vector <int>> result;
```

```
// dfs
for (int i=0; i<v; i++) {
    if (!vis[i]){
        dfs(i, parent, timer, disc,
            low, result, adj);
    }
}
return result;
}
```

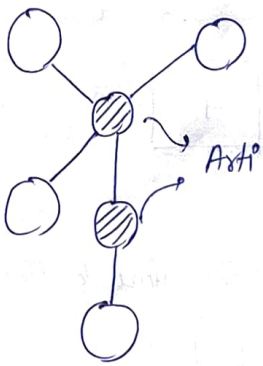
```
void dfs(int node, int parent, &timer,
&disc, &low, &result, adj &, &vis){
    vis[node] = true;
    disc[node] = low[node] = timer++;
    for (auto nbr : adj[node]){
        if (nbr == parent)
            continue;
        if (!vis[nbr]) {
            dfs(nbr, node, timer, disc,
            low, result, adj, vis);
            low[node] = min(low[node],
                low[nbr]);
            // Check bridge condtn
            if (low[nbr] > disc[node]){
                vector<int> ans;
                ans.push_back(node);
                ans.push_back(nbr);
    } } result.push_back(result);
    else {
        // Back edge
        low[node] = min(low[node],
                        disc[nbr]);
    }
}
}
```

T.C = O(V+E) S.C = O(N)

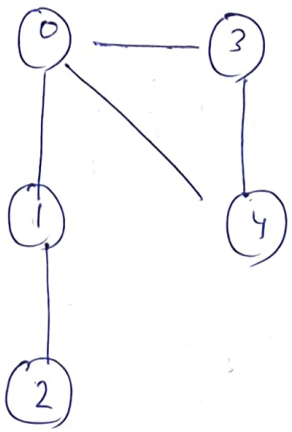
---

lec 99 Articulation point of Graph.

↳ removing that node of graph increases components.



e.g.



disc, low, parent, visited.

condtn.

low[nbr] >= disc[node] &&
parent != -1.

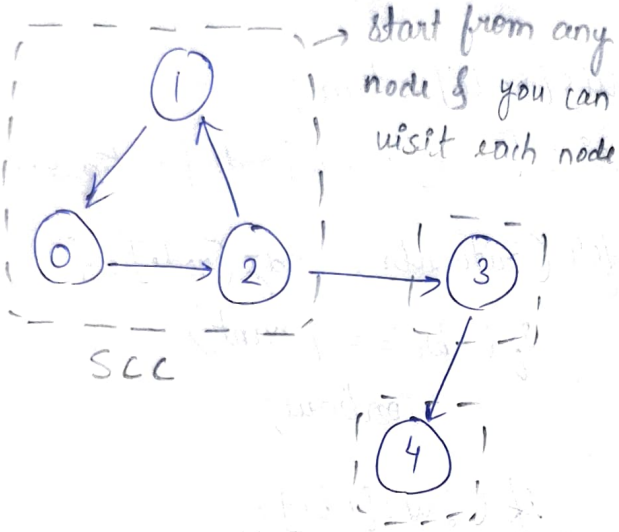
```
int main() {

// adj list
:

---

int timer = 0;
vector<int> disc(n);
"     low(n);
unordered_map<int, bool> vis;
vector<int> ap(n,0);

for (i=0 i<n i++){
    disc[i]= -1;
    low[i]= -1;
}

// dfs
for( int i=0; i<n; i++) {
   if( ! vis[i]) {
      dfs(i, -1, disc, low, vis,
adj, ap, timer);
   }
}

// print ap.
for( int i=0; i<n; i++) {
   if( ap[i]!=0) {
      cout << i << "\n";
   }
}
cout << endl;
```

```
void dfs(int node, int parent,
vector<int> &disc, vector<int> &low
unordered_map<int, bool> &vis, &adj,
&ap, &timer) {

vis[node] = true;
disc[node] = low[node] = timer++;
for(auto nbr: adj[node]) {
   if(nbr == parent)
      continue;
   if(!vis[nbr]){
      dfs(nbr, node, disc, low,
      vis, adj, ap, timer);
      low[node] = min(low[node],
                  low[nbr]);
      // Check ap.
   if( low[nbr] >= disc[node] &&
     parent != -1) {
      ap[node] = true;
   }
   child++;
   }else {
      low[node] = min(low[node],
                  disc[])
   }
}if(parent == -1 && child >1)
```

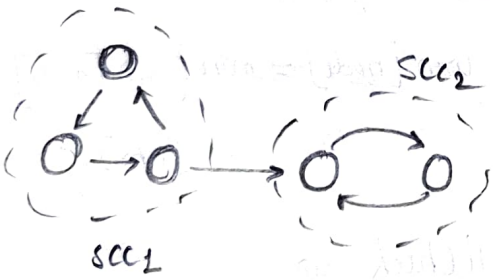
lec100

Kosaraju's Algo

→ count strongly connected components of graph.



sec → [0,1,2], [3], [4].



Algo.

1) sort all nodes basis on their finishing time

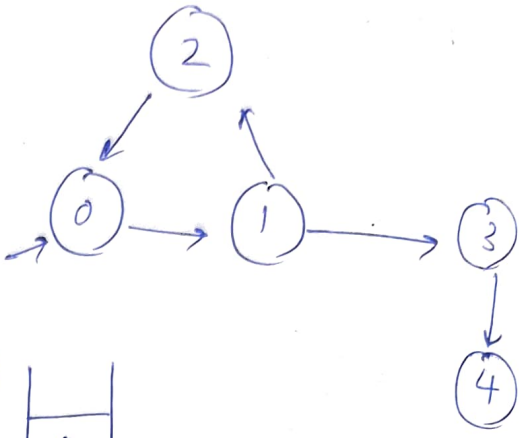
↓

Topological Sort

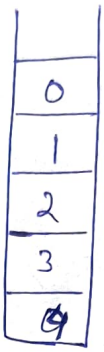
→ Transpose graph

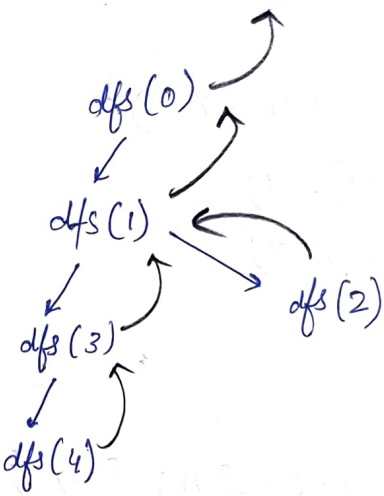
→ dfs → count / print scc.

YOU DON'T VISIT NODE IF ITS ALREADY VISITED.

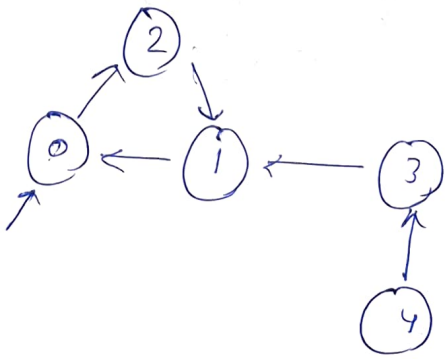


(I)





(II) Transpose graph.



(III) dfs using stack. (topo sort)

code

```
int stronglyConComp (int v,
vector<vector<int>> &edges) {

    // adj list directed graph.
    :
    :

  // topo sort
  stack<int> st;
  unordered_map<int, bool> vis;
  for (int i=0; i<v; i++) {
      if (!vis[i]) {
          dfs (i, vis, st, adj);
      }
  }

// create a transpose graph.
unordered_map<int; list<int>> transpose;
for (int i=0; i<v; i++) {
      vis[i] = 0;
      for (auto nbr: adj[i]){
          transpose[nbr].push_back(i);
      }
}
// dfs call using above ordering (sort).
int count=0
while (!st.empty()) {
      int top = st.top();
        st.pop();
      if (!vis[top]) {
          count++;
        RevDfs (top, vis, transpose);
        }
  }
  return count;
}
```

```
void dfs (node, map &vis,
stack &st, &adj) {
  vis[node] = true;
  for (auto nbr: adj[node]){
      if (!vis[nbr]){
          dfs (nbr, vis, st, adj);
      }
  }
  // topo logic
  st.push(node);
}
```

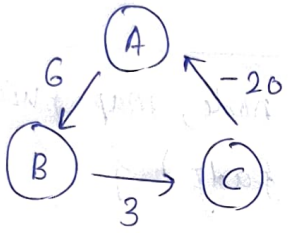
```
void RevDfs (int node, map &vis,
 map &adj)
    vis[node] = true;
    for (auto nbr: adj[node]){
      if (!vis[nbr]){
      RevDfs (nbr, vis, adj);
      }
    }
}
```

Lec 101 **Bellman Ford**

→ directed weighted graph. (may included -ve weighted edges where Dijkstra's algo fails).

→ This algo fails for (-ve) wt cycle.

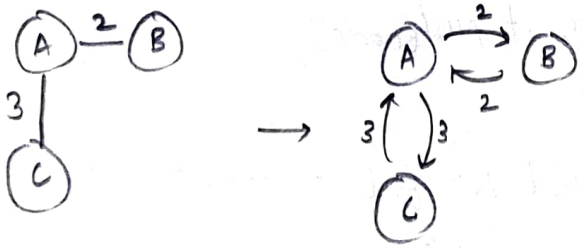
e.g.



sum is < 0 (i.e 6+3-20 = -11)

→ i) Finds (-ve) cycle

ii) if absent then sd finds shortest path.



undirected Graph. directed Graph

① formula is applied (n-1) times on each edge } $u \xrightarrow{wt} v$

```
if ( dist[u] + wt < dist[v] ) {
    dist[v] = dist[u] + wt.
}
// updating dist.
```

② Apply same formula for n^{th} time

dist updates → (-ve cycle present)

No dist updates → (return shortest paths)

```
int bellmanFord( int n, int m, int src,
int desti, vector<vec<int>> &edges){

    vector<int> dist (n+1, 1e9);
    dist[src] = 0;

    for (int i=1; i<=n; i++) {
        for (int j=0; j=m; j++) {
            int u = edges[j][0];
            int v = edges[j][1];
            int wt = edges[j][2];

            if (dist[u] != 1e9 &&
((dist[u] + wt) < dist[v])) {
                dist[v] = dist[u] + wt;
            }
        }
    }
}
```

```
// Check for (-ve) cycle.
bool flag = 0;
for (int j=0; j<m; j++) {
    int u = edges[j][0];
    int v = edges[j][1];
    int wt = edges[j][2];
    if (dist[u] != 1e9 && ((dist[u]
    +wt) < dist[v])) {
        flag = 1;
    }
}
if (flag == 0) {
    return dist[dest];
}
return -1;
}
```